RTM — Requirements Traceability Module

RTM — Requirements Traceability Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Requirements Standard (SRS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Requirements Traceability Governance

Module ID: SLGA-SRS-RTM-005

Origin ID: OOF.CM.SLGA.01.05.01

Version: 1.0

Status: Canonical Draft · Open Standard

Effective Date: Pending Canonical Publication

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Requirements Traceability Module (RTM) governs how software

requirements remain identifiable, relationally connected, evidentially supported, and reconstructable across their origins, purposes, specifications, applicability boundaries, authorized changes, and relevant software lifecycle states.

RTM establishes the methodological conditions under which a relationship

between a requirement and another governance or lifecycle object may be recognized as a traceable, meaningful, and sufficiently supported relationship.

It governs the formation, qualification, preservation, evaluation, and

reassessment of requirements traceability relationships without replacing the competent governance processes responsible for the underlying objects.

RTM applies to software developed, modified, tested, operated, or retired

by humans, organizations, AI agents, autonomous systems, automated development environments, or combinations of these actors.

It preserves the distinction between the existence of a technical link and

the validity of the relationship that the link claims to represent.

Its principal output is:

Governed Requirements Traceability State

Its foundational distinction is:

Recorded Link ≠ Governed Traceability.

Operational Role

The operational role of RTM is:

To establish and preserve evidence-supported relationships that allow software requirements to be followed from their legitimate origins through specification, applicability, change, implementation, verification, and relevant lifecycle outcomes without converting technical associations into unsupported governance conclusions.

RTM makes it possible to determine:

• Why a requirement exists

• Which governed purpose it supports

• Where it applies

• Which changes have affected it

• Which software objects are associated with its implementation

• Which evidence supports its verification or satisfaction state

• Which lifecycle states depend on it

• Whether its traceability remains sufficiently complete, current, and

reconstructable.

RTM does not independently determine that a requirement is legitimate, correctly implemented, or satisfied.

It establishes whether the relationships supporting those determinations

are identifiable and adequately evidenced.

Position in the SRS Internal Governance Flow

The SRS internal governance flow is:

Software Purpose Definition → Requirements Specification → Software Scope

Management → Requirements Change Control → Requirements Traceability

RTM governs:

Requirements Traceability

It is the fifth Core Module of SRS.

Its principal upstream inputs are:

Governed Software Purpose State

from:

SPDM — Software Purpose Definition Module

Governed Software Requirements Specification State

from:

RSPM — Requirements Specification Module

Governed Software Scope State

from:

SSMM — Software Scope Management Module

Governed Requirements Change State

from:

RCCM — Requirements Change Control Module

RTM may also consume relevant governed states and evidence from subsequent SLGA™ lifecycle domains.

Its principal output is:

Governed Requirements Traceability State

That state supports the formation, maintenance, reassessment, and auditability of the:

Governed Software Requirements Baseline

Traceability is established progressively. RTM does not require software implementation, testing, or deployment to have occurred before requirements traceability can begin.

Central Operational Question

Can every material software requirement be reliably connected to its origin, governed purpose, specification, applicability, change history, and relevant lifecycle evidence, with sufficient relationship integrity to reconstruct what was required, why it was required, where it applied, and what became of it?

Requirements Traceability Governance Object

Every RTM assessment must identify the Requirements Traceability Governance Object.

The object represents a requirement, requirements set, or defined

requirements baseline whose relationships are being governed.

It should identify, where applicable:

• Requirement identity

• Requirement version

• Relevant software purpose

• Applicable software scope

• Requirements baseline reference

• Relevant lifecycle context

• Traceability assessment boundary

• Material relationship classes

• Applicable evidence sources

The traceability object shall remain distinguishable from the relationships

associated with it.

Requirement Identity ≠ Traceability Relationship.

A requirement may exist without sufficient traceability.

Likewise, multiple links to a requirement do not independently establish

that its governance relationships are valid.

Traceability Boundary and Purpose

RTM shall establish why traceability is required and which relationships are material to the relevant governance question.

Traceability may support:

• Requirements legitimacy reconstruction

• Purpose alignment

• Requirements completeness assessment

• Scope applicability

• Change-impact assessment

• Implementation association

• Verification and validation evidence

• Release and deployment assessment

• Operational reassessment

• Retirement and historical reconstruction

Not every requirement requires the same number or type of traceability

relationships.

The necessary traceability boundary depends on the requirement's

materiality, applicable obligations, software context, and lifecycle state.

Traceability Volume ≠ Traceability Sufficiency.

Requirement Identity and Reference Integrity

RTM shall preserve stable and distinguishable requirement references.

A requirement reference should identify:

• The requirement

• Its applicable version

• The relevant requirements baseline

• Its identity continuity

• Any material supersession or replacement relationship

The same requirement identifier shall not be used to conceal materially

different requirement meanings.

Conversely, editorial changes shall not automatically require unrelated

requirement identities where continuity can legitimately be preserved.

Identifier Continuity ≠ Meaning Continuity by default.

Changed Wording ≠ New Requirement by default.

Traceability Relationship Object

Every material traceability relationship shall be sufficiently identifiable.

A relationship should preserve:

Source Object: The requirement or governance object from which the

relationship originates.

Target Object: The object to which the relationship refers.

Relationship Type: The meaning of the asserted connection.

Relationship Basis: The evidence or governed state supporting the

connection.

Applicable Context: Relevant software scope, configuration, version, or

lifecycle conditions.

Relationship State: Whether the connection is proposed, supported, conditional, conflicted, invalid, or unresolved.

Temporal Reference: The period or effective state for which the

relationship applies.

A technical hyperlink, database key, or repository reference may represent a relationship.

It does not independently establish its governance validity.

Technical Link ≠ Validated Relationship.

Traceability Direction

RTM shall support traceability in the directions required by the governance

context.

Forward Traceability follows a requirement toward its specification

consequences, implementation associations, verification evidence, and relevant lifecycle states.

Backward Traceability follows a requirement or downstream object toward

its originating need, purpose, authorization basis, and governing requirements state.

Bidirectional Traceability permits reconstruction in both directions

where the underlying relationships support it.

Bidirectional traceability does not require every relationship to have a symmetrical meaning.

For example, a software component may implement a requirement, but the requirement does not thereby implement the component.

Bidirectional Navigation ≠ Symmetrical Relationship Meaning.

Requirement Origin Traceability

RTM shall preserve the relationship between a requirement and its

identifiable origin.

Possible origins include:

• Human requests

• Organizational mandates

• Governed software purposes

• External obligations

• Operational findings

• Existing software requirements

• AI-generated proposals

• Autonomous system observations

The relationship shall distinguish the origin of the requirement from the

authority competent to recognize or accept it.

Origin Traceability ≠ Authorization Evidence.

Where the original source is unknown or reconstructed, RTM shall preserve that limitation.

Software Purpose Traceability

RTM shall establish traceable relationships between requirements and the

applicable:

Governed Software Purpose State

produced by SPDM.

The relationship should identify which intended outcome, purpose condition, or legitimate supporting need the requirement addresses.

A requirement may support multiple purposes.

A purpose may depend on multiple requirements.

Where a requirement has no defensible relationship to the recognized software purpose, RTM shall identify the gap or conflict for competent reassessment.

Purpose Link ≠ Purpose Alignment Validation.

The existence of a link does not establish that the requirement legitimately supports the referenced purpose.

Requirements Specification Traceability

RTM shall preserve relationships between requirement identities and their

governed specification states.

These may include:

• Original requirement expression

• Interpreted meaning

• Current specification

• Material assumptions

• Requirement classification

• Assessment conditions

• Acceptance references

• Previous specification versions

RTM shall preserve the distinction between the original expression and the

accepted or applicable specification.

Original Request ≠ Governed Specification.

Where multiple interpretations exist, the traceability state shall identify the relevant interpretation and its governance condition.

Requirements Decomposition Traceability

Requirements may be decomposed into subordinate requirements.

RTM shall preserve material parent–child relationships.

A decomposition relationship should identify whether a subordinate requirement:

• Refines the parent requirement

• Implements part of its intended behavior

• Establishes a necessary condition

• Represents a derived requirement

• Introduces an additional proposed obligation

RTM shall identify where decomposition leaves material parent requirements without sufficient subordinate coverage.

Requirement Decomposition ≠ Complete Requirement Coverage.

A subordinate requirement shall not inherit legitimacy merely because it is technically linked to an accepted parent.

Derived Requirements Traceability

A derived requirement may originate from analysis, technical constraints, operational assumptions, interface conditions, or other governed requirements.

RTM shall preserve:

• The source requirement or condition

• The derivation rationale

• Material assumptions

• The derived requirement

• The applicable evaluation state

• Relevant authority or acceptance conditions

A technically logical derivation does not automatically establish a binding requirement.

Logical Derivation ≠ Authorized Requirement Creation.

This distinction is especially important where AI systems generate additional requirements from existing specifications.

Software Scope Traceability

RTM shall preserve relationships between requirements and the applicable:

Governed Software Scope State

produced by SSMM.

The relationship should identify where a requirement applies and under which conditions.

Relevant dimensions may include:

• Software system

• Component

• Interface

• Function

• Configuration

• Version

• Operating environment

• Organizational context

• Conditional applicability

A requirement linked to a software system shall not automatically be

treated as applicable to every subordinate component.

System-Level Link ≠ Universal Component Applicability.

Requirements Baseline Traceability

RTM shall preserve relationships between requirements and the governed

baselines in which they are applicable.

A baseline relationship should identify:

• Requirement identity and version

• Baseline identity and version

• Applicable scope

• Acceptance reference

• Effective conditions

• Material exceptions

• Supersession relationships

The same requirement may appear in different baselines under different applicability conditions.

Requirement Inclusion ≠ Universal Baseline Applicability.

RTM shall preserve historical baseline relationships rather than overwrite

them when a later baseline becomes effective.

Requirements Change Traceability

RTM shall preserve relationships between requirements and the applicable:

Governed Requirements Change State

produced by RCCM.

These relationships should permit reconstruction of:

• The previous requirement state

• The proposed change

• The change justification

• Material impact findings

• Applicable authorization conditions

• The change decision

• The effective baseline transition

• The resulting requirement state

A change record shall not be treated as evidence of competent authorization

unless the relevant authorization state supports that conclusion.

Recorded Change ≠ Authorized Change.

Requirements Dependency Traceability

Requirements may depend on other requirements or relevant external conditions.

RTM shall preserve material relationships such as:

Requires

Depends On

Constrains

Conflicts With

Refines

Supersedes

Applies To

Derived From

Relationship types shall remain semantically distinguishable.

For example, a requirement that depends on another requirement is not necessarily a subordinate requirement.

Dependency ≠ Decomposition.

RTM shall not collapse materially different relationship meanings into a

generic association where doing so would undermine governance interpretation.

Software Implementation Traceability

Where software implementation evidence exists, RTM may establish relationships between requirements and relevant software implementation objects.

These may include:

• Source code components

• Services

• Interfaces

• Configuration elements

• Executable functions

• Generated software artifacts

• Relevant implementation records

A requirement may be implemented by multiple software objects.

A software object may contribute to multiple requirements.

RTM shall distinguish an asserted implementation relationship from evidence

that the implementation actually satisfies the requirement.

Code Association ≠ Requirement Satisfaction.

Implemented Functionality ≠ Authorized Requirement.

The Software Creation Standard governs the competent software creation

processes.

Verification Traceability

RTM shall preserve relationships between requirements and relevant

verification evidence.

Such evidence may include:

• Test cases

• Test results

• Inspection records

• Static analysis findings

• Simulation evidence

• Integration assessments

• Competent validation states

The relationship shall distinguish:

Requirement Covered by Test

from:

Test Executed

from:

Test Passed

from:

Requirement Verified

A test may execute successfully without adequately representing the full requirement.

A test may also validate a previous requirement version rather than the currently applicable one.

Passing Test ≠ Verified Requirement by default.

RTM preserves competent verification findings without independently

recreating the verification process.

Validation and Operational Reality Traceability

A software requirement may be correctly implemented and technically verified without supporting the intended operational outcome.

RTM may preserve relationships between requirements and relevant validation or operational reality evidence.

These relationships should distinguish:

• Technical conformance

• Intended-use validation

• Observed operational behavior

• Applicable operating conditions

• Material operational limitations

RTM does not independently certify operational suitability.

Verified Implementation ≠ Validated Operational Outcome.

Where operational evidence contradicts the requirement's intended effect, the relationship shall remain visible for competent reassessment.

Software Quality and Security Traceability

Requirements may relate to software quality, reliability, security, privacy, safety, accessibility, and other material properties.

RTM may preserve relationships to competent governed states from the applicable SLGA™ Parent Standards and external governance domains.

A requirement related to security does not become satisfied merely because a security assessment exists.

The evidence must correspond to the applicable requirement, software object, configuration, and assessment conditions.

Security Evidence Availability ≠ Security Requirement Satisfaction.

RTM does not replace the Software Quality Standard or Software Security Standard.

Configuration and Dependency Traceability

Requirements may apply only to particular configurations or depend on specific software components, libraries, interfaces, or external services.

RTM shall preserve material relationships to these objects where necessary for requirements governance.

Relevant conditions may include:

• Component identity

• Dependency version

• Software configuration

• Runtime environment

• Applicable interface

• External service state

A requirement verified under one configuration shall not automatically be

treated as verified under another.

Configuration Similarity ≠ Evidence Applicability.

Detailed configuration and dependency governance remain within their competent SLGA™ Parent Standards.

Release and Deployment Traceability

Where relevant lifecycle evidence exists, RTM may connect requirements to software release and deployment states.

The relationship should preserve:

• Applicable requirements baseline

• Software release identity

• Relevant configuration

• Deployment environment

• Material verification references

• Applicable exceptions

• Relevant lifecycle status

A requirement may be satisfied in a tested software build without being present in the deployed version.

Verified Build ≠ Deployed Software State.

Likewise, deployment evidence does not independently establish requirements conformance.

Deployment Success ≠ Requirements Satisfaction.

Software Change and Retirement Traceability

Requirements may remain relevant after software has been modified, replaced, suspended, or retired.

RTM shall preserve material relationships between requirements and:

• Software modifications

• Replacement components

• Migration states

• Retired functionality

• Superseded requirements

• Historical baselines

• Relevant continuity obligations

A requirement's historical relevance may survive the retirement of the software component that once implemented it.

Software Retirement ≠ Automatic Requirements Evidence Erasure.

Retention and retirement obligations remain governed by competent lifecycle

and external governance processes.

Traceability Completeness

RTM shall assess whether all materially required relationship classes are sufficiently represented for the applicable traceability purpose.

Completeness shall be evaluated relative to:

• The governed software purpose

• Requirement materiality

• Applicable software scope

• Lifecycle stage

• Relevant external obligations

• Required evidence relationships

• Applicable governance questions

Traceability completeness does not require every possible link.

It requires sufficient coverage of the relationships necessary to

reconstruct the relevant requirements governance state.

Link Quantity ≠ Traceability Completeness.

Traceability Coverage

RTM shall identify which material requirements and relationship dimensions

are:

Sufficiently Traced

Partially Traced

Untraced

Unresolved

Coverage may be evaluated across:

• Requirement origins

• Purpose relationships

• Specification states

• Scope applicability

• Change history

• Implementation associations

• Verification evidence

• Relevant lifecycle states

Strong traceability in one dimension shall not conceal material gaps in

another.

Implementation Traceability ≠ Complete Requirements Traceability.

Traceability Gap

A Traceability Gap exists where a materially necessary relationship is missing, unsupported, outdated, or insufficiently defined.

Examples include:

• A requirement without an identifiable origin

• A requirement without a defensible purpose relationship

• An implemented feature without a recognized requirement

• A requirement without applicable scope information

• A baseline transition without a competent change reference

• A claimed verification state without supporting evidence

• A deployed capability without a reconstructable requirements reference

RTM shall identify the gap, affected requirement, missing relationship, and

relevant governance consequence.

Missing Link ≠ Missing Requirement by default.

The absence of a relationship may reflect missing documentation, missing evidence, or a substantive governance defect.

These possibilities shall remain distinguishable.

Critical Traceability Gap

A Critical Traceability Gap exists where missing or unreliable relationships prevent a materially consequential requirement state from being sufficiently reconstructed or evaluated.

Examples may include:

- An autonomous financial action without a traceable authorized

requirement. - A safety-critical software function without identifiable verification evidence. - A requirements baseline change without a competent authorization reference. - A deployed capability whose applicable purpose and scope cannot be established.

A critical gap may require:

• Additional evidence acquisition

• Relationship reconstruction

• Requirements reassessment

• Competent validation

• Conditional governance status

• Escalation to the responsible governance domain

RTM shall not independently determine the operational response where that decision belongs to another competent process.

Traceability Evidence and Source Competence

Every material traceability claim should be supported by a source competent to establish the asserted relationship.

Examples include:

• SPDM for governed software purpose states

• RSPM for governed specification states

• SSMM for governed scope applicability

• RCCM for governed requirements changes

• Competent software creation systems for implementation records

• Competent verification processes for verification results

• Competent release and deployment processes for their respective

lifecycle states.

A repository may establish that a file was modified.

It does not necessarily establish that the modification was authorized.

A test system may establish that a test passed.

It does not necessarily establish that the requirement was fully satisfied.

Evidence Source Competence ≠ Universal Governance Competence.

Traceability Provenance

RTM shall preserve the provenance of material traceability relationships.

Relevant provenance may include:

• Relationship origin

• Source and target references

• Supporting evidence

• Relationship creation method

• Applicable governance source

• Relevant actor or system

• Version and effective context

• Material modifications

Where a relationship was inferred by an AI system, the inference shall remain distinguishable from a relationship established by competent evidence or recognition.

AI-Inferred Relationship ≠ Evidentially Established Relationship.

Traceability Confidence and Uncertainty

Traceability relationships may differ in evidentiary strength.

RTM shall preserve material uncertainty concerning:

• Source identity

• Relationship meaning

• Evidence sufficiency

• Configuration applicability

• Version alignment

• Historical reconstruction

• Conflicting records

• Automated inference

Where confidence is represented, it shall remain connected to the supporting evidence and assessment method.

A high statistical similarity score does not independently establish a valid governance relationship.

Semantic Similarity ≠ Traceability Validity.

Traceability Conflict

Traceability records may contain contradictory relationships.

Examples include:

• Two requirements claiming incompatible origins

• Conflicting scope applicability records

• Different baseline versions presented as current

• Implementation evidence linked to the wrong requirement version

• Verification results associated with an incompatible configuration

• Conflicting claims about the authority for a requirements change

RTM shall preserve the conflicting relationships and their evidence.

Where reconciliation is possible, the basis shall remain traceable.

Where competent resolution is unavailable, the relationship state shall remain conflicted or unresolved.

Relationship Conflict ≠ Permission to Select the Preferred Evidence.

Traceability Duplication and Independence

The same underlying relationship may be repeated across multiple tools, repositories, reports, or automated analyses.

RTM shall distinguish repeated representation from independent supporting

evidence.

For example, several AI-generated traceability reports may rely on the same original repository link.

Their agreement does not necessarily provide independent confirmation.

Repeated Link ≠ Independent Evidence.

RTM may identify corroboration where materially independent sources support the same relationship.

Traceability Temporal and Version Alignment

A relationship may have been valid at one time but no longer represent the current requirements or software state.

RTM shall preserve material alignment between:

• Requirement versions

• Requirements baselines

• Software versions

• Configuration versions

• Verification evidence

• Relevant lifecycle states

• Effective dates or periods

Historical relationships shall remain reconstructable without being

misrepresented as current.

Historical Traceability ≠ Current Traceability.

Traceability Drift

Traceability Drift occurs when previously supported relationships no longer adequately represent the current requirements or software reality.

Possible causes include:

• Requirements changes

• Software refactoring

• Component replacement

• Configuration changes

• New dependencies

• Modified interfaces

• Changes in test coverage

• Deployment changes

• Autonomous software modifications

• Retirement of relevant software objects

RTM shall identify material drift and trigger reassessment of affected

relationships.

Previously Valid Link ≠ Permanently Valid Link.

Traceability Under Autonomous Development

AI agents and autonomous development systems may create, modify, or infer traceability relationships.

RTM shall preserve the distinction between:

Observed Technical Association

Machine-Inferred Relationship

Evidence-Supported Relationship

Governed Traceability Relationship

An agent may correctly identify that a code function appears related to a requirement.

However, this does not independently establish:

• That the requirement was authorized

• That the code implements the requirement completely

• That the relationship applies to the current configuration

• That the requirement has been verified

• That the agent was authorized to create or modify the underlying

software object.

Automated Trace Generation ≠ Governed Traceability.

Task Authority ≠ Constituent Action Authority.

RTM may consume AI-generated relationship proposals while preserving their provisional status until the applicable governance conditions are satisfied.

Traceability Across Multiple Agents and Tools

A software requirement may be processed through multiple agents, repositories, development tools, testing systems, and deployment environments.

RTM shall preserve material relationship continuity across these systems.

The traceability chain may include:

Governed Requirement → Agent Task → Tool Action → Software Artifact →

Verification Evidence → Lifecycle State

Where tool actions materially affect a requirement or its implementation evidence, the relationship should remain reconstructable.

However, the existence of an authorized task does not establish that every downstream tool action was authorized.

RTM preserves relevant authority references without replacing the competent

agent and tool-action governance processes.

Task Traceability ≠ Tool-Action Authorization.

Traceability Reconstruction for Legacy Software

Existing software may lack complete historical requirements records.

RTM shall support reconstruction where the original relationships are incomplete or unavailable.

Reconstruction may use:

• Historical specifications

• Source code

• Version-control records

• Test records

• Operational documentation

• Change histories

• Relevant organizational records

• Competent technical assessments

Reconstructed relationships shall distinguish:

Historically Evidenced

Reasonably Inferred

Unverified

Unknown

Reconstruction shall not manufacture missing historical authorization or

present inferred relationships as verified historical facts.

Reconstructed Traceability ≠ Recovered Historical Authority.

Traceability Integrity

RTM shall preserve traceability relationships against material distortion.

Relevant risks include:

• Unsupported link creation

• Silent link removal

• Incorrect relationship classification

• Misleading version association

• Selective omission

• Evidence substitution

• Automated overstatement of confidence

• Retrospective alteration of change history

• Concealment of unresolved gaps

Traceability integrity requires that the recorded relationships reflect the available evidence and governed states rather than a preferred conclusion.

Traceability Integrity ≠ Traceability Completeness.

A complete-looking traceability matrix may still contain invalid relationships.

Traceability Matrix and Relationship Graph

RTM may represent relationships through a:

Requirements Traceability Matrix

or:

Requirements Traceability Graph

or another suitable representation.

A matrix may be useful for comparing requirements against defined relationship dimensions.

A graph may be useful for representing complex dependencies, many-to-many relationships, and lifecycle connections.

Neither format is universally mandatory.

The representation shall preserve:

• Requirement identity

• Relationship type

• Source and target

• Applicable context

• Evidence

• Governance state

• Material uncertainty

Traceability Visualization ≠ Traceability Governance.

A visually complete diagram does not establish that its relationships are valid.

Traceability Baseline

RTM may establish a traceability baseline corresponding to a defined requirements baseline and lifecycle context.

The baseline should preserve:

• Applicable requirement versions

• Relationship states

• Relevant software scope

• Supporting evidence

• Material gaps

• Configuration and temporal context

• Applicable methodology version

Later traceability states may be compared against this reference.

The traceability baseline shall not overwrite the requirements baseline or

independently determine its legitimacy.

Traceability Change and Impact

Where a requirement or linked lifecycle object changes, RTM shall identify relationships potentially affected by the change.

Examples include:

• A changed requirement affecting multiple tests

• A replaced component affecting several requirements

• A modified interface invalidating previous verification evidence

• A revised scope changing requirements applicability

• A new deployment configuration requiring reassessment of earlier

evidence.

RTM shall support impact discovery without independently authorizing the

resulting requirements or software changes.

Traceability Impact Identification ≠ Change Authorization.

Traceability Continuity

Requirements traceability shall remain reconstructable across material

changes in:

• Development actors

• Organizations

• Software suppliers

• AI agents

• Tooling

• Repositories

• Software versions

• Configurations

• Operating environments

• Relevant lifecycle states

Continuity does not require every historical system to remain operational.

It requires preservation of sufficient identifiers, relationships, evidence

references, and historical states to reconstruct the material requirements governance chain.

Tool Continuity ≠ Traceability Continuity.

Traceability Governance States

RTM shall represent the governance condition of material traceability

relationships and the overall traceability object.

Possible states include:

Proposed: A relationship has been asserted or generated but not

sufficiently evaluated.

Under Assessment: The relationship is undergoing evidence and applicability assessment.

Supported: The relationship has sufficient evidence for its defined

governance use.

Conditionally Supported: The relationship remains dependent on explicit

conditions.

Partially Traced: Material relationship coverage remains incomplete.

Evidence-Limited: Supporting evidence is insufficient.

Conflicted: Material relationship contradictions remain unresolved.

Stale: The relationship no longer has sufficiently current support.

Invalid: The relationship has been determined not to represent the

claimed connection.

Superseded: A later relationship state has replaced the previous state.

Unresolved: The available information does not support a sufficiently determined relationship state.

An overall traceability state may contain relationships with different individual states.

The overall state shall not conceal material gaps or conflicts through aggregation.

Supported Relationship ≠ Verified Requirement.

Traceability Readiness

RTM shall assess whether the available traceability is sufficiently complete, current, coherent, and evidenced for the relevant governance use.

Traceability readiness may differ by purpose.

For example:

• Requirements review may require origin, purpose, specification, and

• Change-impact assessment may require dependencies and baseline history

• Release assessment may require implementation and verification

• Retirement assessment may require historical obligations and

scope relationships. relationships. continuity relationships.

Traceability readiness shall not be represented as a universal binary

condition independent of the governance question.

Traceability Readiness ≠ Software Readiness.

A traceability state may be sufficient for one assessment and insufficient for another.

Traceability Reassessment

RTM shall support reassessment when:

• Requirements change

• Purpose or scope changes

• New evidence becomes available

• Existing evidence becomes stale

• Software configurations change

• Implementation objects are replaced

• Verification findings change

• New lifecycle relationships emerge

• Previously inferred relationships are challenged

• Material gaps or conflicts are discovered

Reassessment shall preserve previous relationship states where necessary for continuity and reconstruction.

Governed Requirements Traceability State

The principal output of RTM is: Governed Requirements Traceability State

The state shall preserve sufficient information to identify:

Traceability Governance Object: The requirements or baseline whose

relationships are governed.

Requirements Identity: Applicable requirement identifiers and versions.

Traceability Boundary: The governance purpose and relationship coverage

required.

Origin Relationships: Connections to requirement sources.

Purpose Relationships: Connections to governed software purpose states.

Specification Relationships: Connections to governed requirement

specifications.

Scope Relationships: Connections to applicable software scope states.

Change Relationships: Connections to governed requirements change states.

Baseline Relationships: Connections to applicable requirements baselines.

Dependency Relationships: Material requirement and software dependencies.

Implementation Relationships: Relevant software artifact associations.

Verification Relationships: Supporting verification evidence and

limitations.

Validation Relationships: Relevant competent validation states.

Lifecycle Relationships: Applicable configuration, release, deployment,

change, and retirement references.

Relationship Types: The meaning of material connections.

Evidence and Provenance: Supporting sources and their competence.

Coverage: Sufficiently traced, partially traced, untraced, and unresolved

dimensions.

Traceability Gaps: Material missing or unsupported relationships.

Conflicts: Contradictory or irreconcilable relationships.

Uncertainty: Material evidentiary limitations.

Temporal and Version Context: Relevant effective periods and

configurations.

Integrity: Material risks affecting relationship validity.

Governance States: Current relationship and overall traceability

conditions.

Continuity: Historical relationships, supersession, and reassessment

history.

The state shall not independently imply requirements acceptance,

implementation conformance, software safety, release readiness, or deployment authorization.

Minimum Implementation Framework

A conforming implementation of RTM must provide the following six functional capabilities.

• Requirements Identity and Traceability Boundary Governance

Capability to identify requirements, versions, relevant baselines,

traceability objectives, and the material relationship classes required for the applicable governance context.

The implementation must distinguish requirement identity from relationship identity and preserve relevant temporal and configuration context.

• Multi-Domain Relationship Formation and Qualification Governance

Capability to establish and qualify relationships connecting requirements

to origin, purpose, specification, scope, changes, baselines, dependencies, implementation objects, verification evidence, and relevant lifecycle states.

The implementation must preserve relationship meaning and distinguish technical associations from governed relationships.

• Traceability Coverage, Gap, and Readiness Governance

Capability to assess material relationship completeness, coverage, critical

gaps, and traceability readiness relative to the applicable governance question.

The implementation must not infer completeness from link quantity or visual matrix coverage.

• Relationship Evidence, Provenance, and Integrity Governance

Capability to preserve evidence, source competence, provenance,

relationship states, uncertainty, conflicts, version alignment, and integrity conditions.

The implementation must distinguish machine-inferred relationships from sufficiently evidenced relationships.

• Traceability Change, Drift, and Continuity Governance

Capability to identify relationships affected by requirements or software

changes, detect stale or invalid links, preserve historical states, support legacy reconstruction, and trigger reassessment.

The implementation must not erase material historical relationships when current states change.

• Governed Traceability State and Lifecycle Handoff Governance

Capability to produce a Governed Requirements Traceability State

sufficiently explicit for SRS baseline governance and competent downstream SLGA™ lifecycle processes.

The implementation must preserve unresolved gaps, conflicts, uncertainty, and applicability conditions during handoff.

GIE™ Compatibility

RTM is compatible with GIE™ as a downstream methodological execution and analysis layer.

GIE™ may support:

• Requirements relationship discovery

• Structured traceability mapping

• Purpose and scope relationship analysis

• Requirements dependency analysis

• Change-impact relationship discovery

• Implementation and verification evidence association

• Traceability completeness assessment

• Critical gap detection

• Version and configuration alignment

• Traceability drift detection

• Legacy relationship reconstruction

• Evidence and provenance mapping

• Governance state representation

• Reassessment and lifecycle handoff

GIE™ may propose traceability relationships using automated analysis, including semantic or structural inference.

Such proposals shall retain their evidentiary and governance status.

GIE™ must not convert inferred relationships into validated facts, create missing historical authorization, or represent traceability completeness as proof of software conformance.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical RTM governance logic is:

Receive Governed Software Purpose State

↓

Receive Governed Software Requirements Specification State

↓

Receive Governed Software Scope State

↓

Receive Governed Requirements Change State

↓

Identify Requirements Traceability Governance Object

↓

Identify Applicable Requirements Baseline

↓

Define Traceability Boundary and Governance Question

↓

Identify Requirement Identities and Versions

↓

Identify Material Relationship Classes

↓

Establish Origin and Purpose Relationships

↓

Establish Specification and Scope Relationships

↓

Establish Change and Baseline Relationships

↓

Identify Material Dependencies

↓

Identify Relevant Implementation Relationships

↓

Identify Relevant Verification and Validation Relationships

↓

Identify Applicable Lifecycle Relationships

↓

Assess Relationship Meaning and Source Competence

↓

Verify Temporal, Version, Scope, and Configuration Alignment

↓

Assess Evidence, Provenance, and Uncertainty

↓

Assess Traceability Completeness and Coverage

↓

Identify Gaps and Critical Traceability Gaps

↓

Identify Conflicts, Duplication, and Unsupported Inference

↓

Assess Relationship Integrity and Drift

↓

Determine Traceability Governance States

↓

Establish Governed Requirements Traceability State

↓

Preserve Historical Relationships and Continuity

↓

Support SRS Baseline Governance and SLGA™ Lifecycle Handoff

↓

Reassess When Material Requirements or Lifecycle Conditions Change

The principal output is:

Governed Requirements Traceability State

Governance Outputs

RTM may produce:

Requirements Traceability Governance Object

Traceability Boundary Record

Requirements Identity Map

Requirements Version Reference

Traceability Relationship Record

Relationship Type Record

Requirement Origin Trace

Software Purpose Trace

Requirements Specification Trace

Requirements Decomposition Trace

Derived Requirements Trace

Software Scope Trace

Requirements Baseline Trace

Requirements Change Trace

Requirements Dependency Trace

Software Implementation Trace

Verification Evidence Trace

Validation Evidence Trace

Configuration Trace

Software Dependency Trace

Release Trace

Deployment Trace

Software Change Trace

Retirement Trace

Traceability Completeness Assessment

Traceability Coverage State

Traceability Gap Record

Critical Traceability Gap

Relationship Evidence Record

Relationship Provenance Record

Relationship Confidence and Uncertainty State

Relationship Conflict Record

Relationship Duplication Record

Temporal and Version Alignment State

Traceability Drift Record

Traceability Integrity State

Requirements Traceability Matrix

Requirements Traceability Graph

Traceability Baseline

Traceability Change Impact Record

Legacy Traceability Reconstruction Record

Traceability Continuity Record

Traceability Governance State

Traceability Readiness State

Traceability Reassessment Record

Lifecycle Handoff Record

The principal output is:

Governed Requirements Traceability State

Critical Distinctions

RTM preserves the following distinctions:

Recorded Link ≠ Governed Traceability

Requirement Identity ≠ Traceability Relationship

Traceability Volume ≠ Traceability Sufficiency

Technical Link ≠ Validated Relationship

Bidirectional Navigation ≠ Symmetrical Relationship Meaning

Origin Traceability ≠ Authorization Evidence

Purpose Link ≠ Purpose Alignment Validation

Original Request ≠ Governed Specification

Requirement Decomposition ≠ Complete Requirement Coverage

Logical Derivation ≠ Authorized Requirement Creation

System-Level Link ≠ Universal Component Applicability

Requirement Inclusion ≠ Universal Baseline Applicability

Recorded Change ≠ Authorized Change

Dependency ≠ Decomposition

Code Association ≠ Requirement Satisfaction

Implemented Functionality ≠ Authorized Requirement

Passing Test ≠ Verified Requirement by default

Verified Implementation ≠ Validated Operational Outcome

Security Evidence Availability ≠ Security Requirement Satisfaction

Configuration Similarity ≠ Evidence Applicability

Verified Build ≠ Deployed Software State

Deployment Success ≠ Requirements Satisfaction

Software Retirement ≠ Automatic Requirements Evidence Erasure

Link Quantity ≠ Traceability Completeness

Implementation Traceability ≠ Complete Requirements Traceability

Missing Link ≠ Missing Requirement by default

Evidence Source Competence ≠ Universal Governance Competence

AI-Inferred Relationship ≠ Evidentially Established Relationship

Semantic Similarity ≠ Traceability Validity

Relationship Conflict ≠ Permission to Select the Preferred Evidence

Repeated Link ≠ Independent Evidence

Historical Traceability ≠ Current Traceability

Previously Valid Link ≠ Permanently Valid Link

Automated Trace Generation ≠ Governed Traceability

Task Traceability ≠ Tool-Action Authorization

Reconstructed Traceability ≠ Recovered Historical Authority

Traceability Integrity ≠ Traceability Completeness

Traceability Visualization ≠ Traceability Governance

Traceability Impact Identification ≠ Change Authorization

Tool Continuity ≠ Traceability Continuity

Supported Relationship ≠ Verified Requirement

Traceability Readiness ≠ Software Readiness

Cross-Architecture Boundary

RTM governs requirements traceability within SRS and SLGA™.

It does not replace the governance domains responsible for the underlying requirements, software artifacts, verification states, operational evidence, authority, or lifecycle decisions.

Within SRS:

SPDM — Software Purpose Definition Module governs software purpose

definition.

RSPM — Requirements Specification Module governs requirements

specification and completeness assessment.

SSMM — Software Scope Management Module governs requirements scope and

applicability.

RCCM — Requirements Change Control Module governs requirements changes

and baseline transitions.

RTM — Requirements Traceability Module governs the relationships

connecting these states to one another and to relevant software lifecycle evidence.

Within SLGA™:

Software Creation Standard governs software creation.

Software Dependency Standard governs software dependencies.

Software Configuration Standard governs software configuration.

Software Quality Standard governs software quality.

Software Security Standard governs software security.

Software Release Standard governs software release.

Software Deployment Standard governs software deployment.

Software Change Management Standard governs software lifecycle changes.

Software Retirement Standard governs software retirement.

Software Lifecycle Control Standard governs cross-cutting lifecycle

control.

Relevant interoperating OOF® architectures may include:

GOA™ — Governance Architecture: General authority, mandate, and

delegation boundaries.

AGA™ — Accountability Governance Architecture: Applicable responsibility

and accountability conditions.

ASGA™ — Autonomous Systems Governance Architecture: Autonomous system

authority and execution boundaries.

ORA™ — Operational Reality Architecture: Competent operational reality

assessment.

VALIDOS™: Applicable validation methodology.

ADIT®: Broader evidence and auditability governance.

RTM may consume relevant governed states from these domains to establish traceability relationships.

It does not independently recreate or replace their governance determinations.

Canonical Boundary

RTM begins when a software requirement or requirements baseline requires identifiable, meaningful, and evidence-supported relationships to its origin, purpose, specification, applicability, change history, or relevant software lifecycle objects.

RTM governs:

• Requirements traceability objects

• Traceability boundaries

• Requirement identity and version references

• Traceability relationship objects

• Relationship meaning and direction

• Requirement origin traceability

• Software purpose traceability

• Requirements specification traceability

• Requirements decomposition

• Derived requirements

• Software scope traceability

• Requirements baseline traceability

• Requirements change traceability

• Requirements dependencies

• Software implementation relationships

• Verification and validation relationships

• Software quality and security relationships

• Configuration and dependency relationships

• Release and deployment relationships

• Software change and retirement relationships

• Traceability completeness

• Traceability coverage

• Traceability gaps

• Critical Traceability Gaps

• Relationship evidence and provenance

• Relationship uncertainty and conflict

• Relationship duplication and independence

• Temporal and version alignment

• Traceability drift

• Autonomous traceability generation

• Multi-agent and tool-chain traceability

• Legacy traceability reconstruction

• Traceability integrity

• Traceability matrices and graphs

• Traceability baselines

• Traceability change impacts

• Traceability continuity

• Traceability governance states

• Traceability readiness

• Traceability reassessment

RTM does not independently determine:

• Requirements legitimacy

• Software purpose recognition

• Requirements specification acceptance

• Requirements scope authorization

• Requirements change authorization

• Software implementation conformance

• Software security or quality certification

• Software release readiness

• Deployment authorization

• Operational safety

• General legal authority or accountability

RTM ends when the relevant requirements relationships have been represented as a:

Governed Requirements Traceability State

with sufficient identity, relationship meaning, evidence, coverage,

integrity, temporal context, and continuity to support SRS requirements baseline governance and competent downstream SLGA™ lifecycle processes.

• Use Case 1 — Every Test Passes, but Nobody Can Prove What Was Required

A software company develops a payment-processing application.

Its automated testing dashboard reports that all tests pass.

The development team considers the software ready for release.

However, a requirements review identifies that several tests are not linked to the currently accepted requirements baseline.

Some tests were created for an earlier software version.

Others verify technical behavior without establishing which governed

requirement that behavior is intended to satisfy.

RTM reconstructs the relevant relationships:

Governed Requirement

↓

Applicable Requirements Baseline

↓

Software Implementation Object

↓

Test Case

↓

Test Result

↓

Verification Evidence State

The assessment identifies:

• Requirements without sufficient test relationships

• Tests linked to superseded requirements

• Version mismatches

• Missing acceptance references

• Requirements whose verification status cannot be established

The resulting traceability state may remain:

Partially Traced

or:

Evidence-Limited

even though the automated testing dashboard is green.

Passing Test ≠ Verified Requirement by default.

Test Coverage ≠ Requirements Traceability Completeness.

RTM prevents a technically successful testing process from being mistaken

for a sufficiently governed requirements evidence chain.

Use Case 2 — An AI Agent Produces Working Code Without a Traceable

Authorized Requirement

An organization uses multiple AI coding agents to maintain an internal

business application.

One agent analyzes user feedback.

Another proposes changes.

A third generates code.

A fourth creates automated tests.

The resulting software feature works correctly and passes its tests.

However, the feature introduces a new automated action that was not present in the recognized requirements baseline.

The development pipeline contains:

• Agent execution records

• Code changes

• Successful test results

• Deployment artifacts

What it lacks is a traceable connection to a legitimately accepted and applicable requirement.

RTM reconstructs the available chain:

Agent Proposal → Generated Requirement → Code Artifact → Test Evidence →

Deployment State

It then identifies the missing governance relationship:

Accepted and Applicable Requirement → Authorized Software Capability

The feature's technical traceability may be extensive.

Its requirements legitimacy remains unresolved.

RTM records the gap without inventing the missing authorization.

The appropriate competent processes may then evaluate the requirement, scope, change, and deployment implications.

Technical Execution Trace ≠ Governed Requirements Traceability.

Working Code ≠ Authorized Software Capability.

Task Authority ≠ Constituent Action Authority.

RTM ensures that increasingly autonomous software development remains connected to the requirements governance chain rather than merely to a sequence of successful tool executions.

Canonical Closing Statement

Software requirements do not remain governable merely because they are documented.

They must remain connected to the purposes that justify them, the specifications that define them, the scopes in which they apply, and the changes that alter their meaning or obligations.

As software moves through creation, configuration, verification, release, deployment, modification, and retirement, those relationships become increasingly important.

A repository can record a code change without establishing its authorization.

A test can pass without proving that the correct requirement was tested.

A deployment can succeed without demonstrating that the deployed capability belongs to the accepted requirements baseline.

And an AI agent can generate thousands of technically plausible links without establishing that those links represent valid governance relationships.

Recorded Link ≠ Governed Traceability.

Code Association ≠ Requirement Satisfaction.

Passing Test ≠ Verified Requirement by default.

AI-Inferred Relationship ≠ Evidentially Established Relationship.

Technical Execution Trace ≠ Governed Requirements Traceability.

Traceability Readiness ≠ Software Readiness.

RTM establishes the methodological conditions under which requirements

relationships become identifiable, meaningful, evidenced, current, and reconstructable.

It preserves missing links rather than inventing them.

It preserves historical relationships without presenting them as current.

It distinguishes technical association from competent governance evidence.

And it maintains continuity when software, actors, development tools, organizations, and autonomous systems change.

RTM does not determine whether the software should be released or deployed.

It establishes whether the requirements governance relationships needed to

support those decisions can be reliably reconstructed.

RTM transforms software requirements traceability from a collection of

technical links into a governed lifecycle evidence structure — ensuring that every material software requirement can remain connected to its origin, purpose, applicability, authorized evolution, and operational consequences without allowing recorded associations to substitute for legitimate governance evidence.